

Relaciones Uno a Muchos (1:N)

Objetivos de Aprendizaje

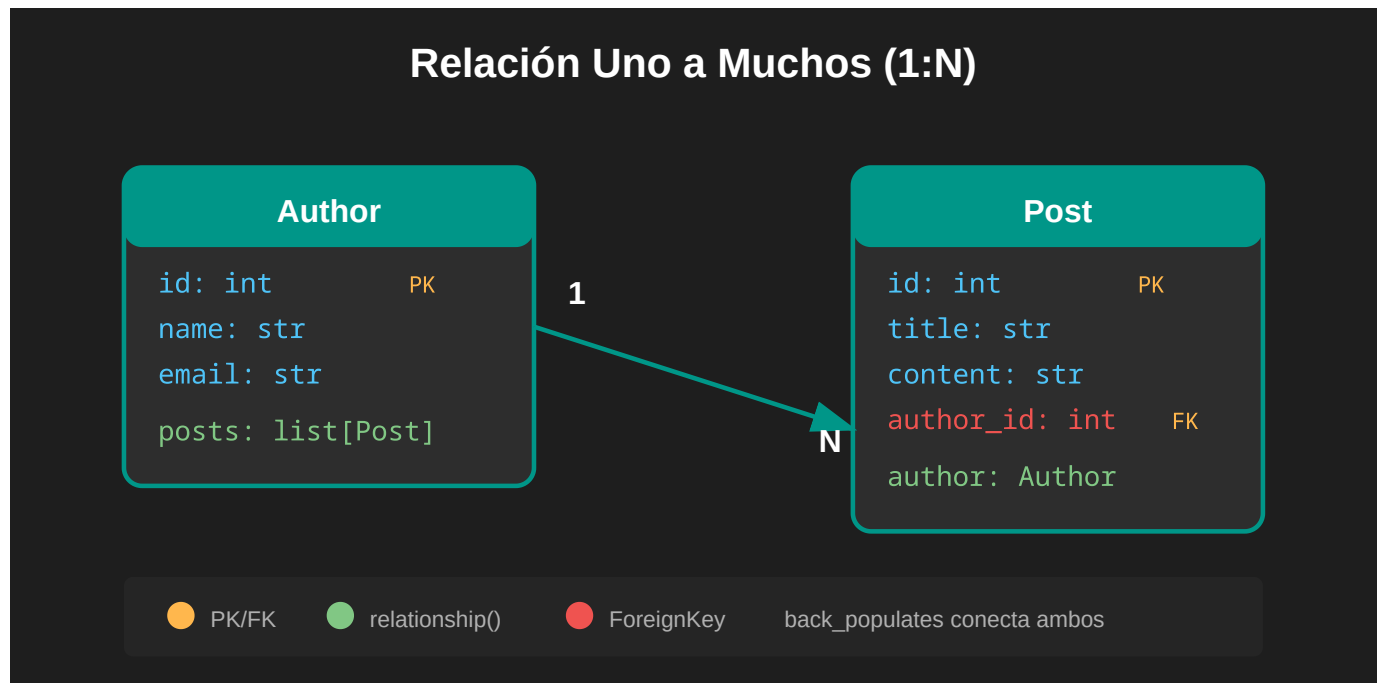
Al finalizar este tema, serás capaz de:

-  Entender qué es una relación uno a muchos
-  Implementar `ForeignKey` en SQLAlchemy
-  Usar `relationship()` para navegar entre modelos
-  Configurar `back_populates` para relaciones bidireccionales

Contenido

1. ¿Qué es una Relación 1:N?

Una relación **uno a muchos** conecta un registro de una tabla con múltiples registros de otra:



Ejemplos del mundo real:

- Un **Author** tiene muchos **Posts** (1 autor → N posts)
- Un **User** tiene muchas **Orders** (1 usuario → N pedidos)
- Un **Department** tiene muchos **Employees** (1 depto → N empleados)

2. Implementación en SQLAlchemy

Paso 1: Definir el lado "Uno" (Parent)

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Author(Base):
    __tablename__ = "authors"
```

```

id: Mapped[int] = mapped_column(primary_key=True)
name: Mapped[str] = mapped_column(String(100))

# Relación: Un autor tiene muchos posts
posts: Mapped[list["Post"]] = relationship(
    back_populates="author"
)

```

Paso 2: Definir el lado "Muchos" (Child)

```

from sqlalchemy import String, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str]

    # Foreign Key: referencia al autor
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

    # Relación inversa: cada post pertenece a un autor
    author: Mapped["Author"] = relationship(
        back_populates="posts"
    )

```

3. Componentes Clave

ForeignKey

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

- Define la **columna** que almacena la referencia
- El string "authors.id" es "nombre_tabla.columna"
- Se coloca en el lado "**muchos**" (Post)

relationship()

```
posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

- Define cómo **navegar** entre objetos Python
- **NO crea columnas** en la base de datos
- Mapped[list["Post"]] → un autor tiene una **lista** de posts
- Mapped["Author"] → un post tiene **un** autor

back_populates

Conecta ambos lados de la relación:

```
# En Author
posts: Mapped[list["Post"]] = relationship(back_populates="author")

# En Post
author: Mapped["Author"] = relationship(back_populates="posts")
```

- Los nombres deben coincidir cruzados
- Permite navegar en **ambas direcciones**

4. Usando la Relación

Crear Post con Author

```
from sqlalchemy.orm import Session

def create_post_for_author(
    db: Session,
    author_id: int,
    title: str,
    content: str
) -> Post:
    post = Post(
        title=title,
        content=content,
        author_id=author_id # Solo necesitas el ID
    )
    db.add(post)
    db.commit()
    db.refresh(post)
    return post
```

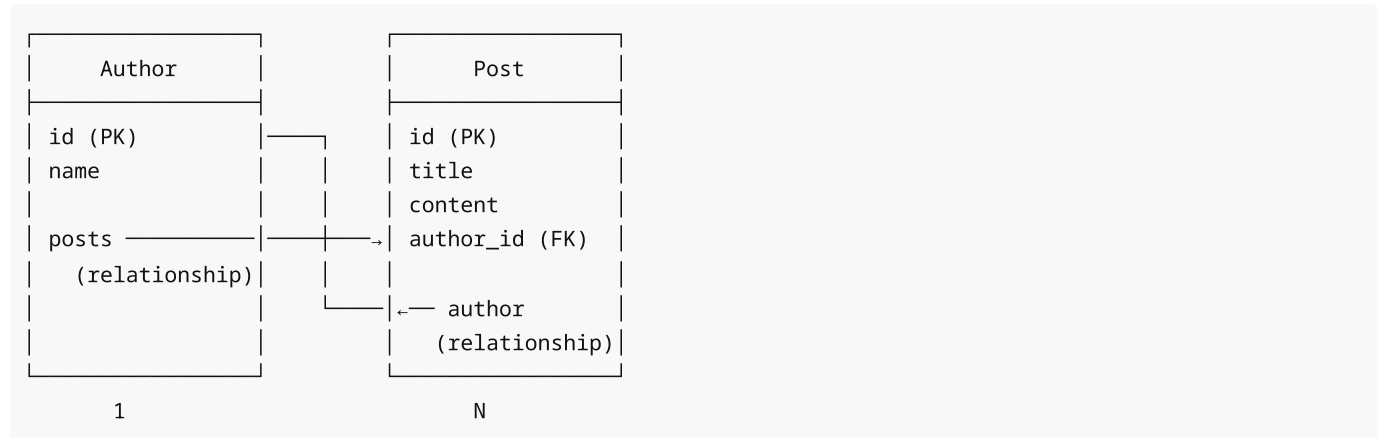
Navegar de Author a Posts

```
def get_author_with_posts(db: Session, author_id: int) -> Author | None:
    author = db.get(Author, author_id)
    if author:
        # Acceder a los posts es automático
        print(f"Author: {author.name}")
        print(f"Posts: {len(author.posts)}")
        for post in author.posts:
            print(f"  - {post.title}")
    return author
```

Navegar de Post a Author

```
def get_post_with_author(db: Session, post_id: int) -> Post | None:
    post = db.get(Post, post_id)
    if post:
        # Acceder al autor es automático
        print(f"Post: {post.title}")
        print(f"Author: {post.author.name}")
    return post
```

5. Diagrama de la Relación



6. Opciones Comunes de relationship()

```
# Cascade delete: eliminar posts cuando se elimina el autor
posts: Mapped[list["Post"]] = relationship(
    back_populates="author",
    cascade="all, delete-orphan"
)

# Lazy loading (por defecto): carga posts cuando accedes
posts: Mapped[list["Post"]] = relationship(
    back_populates="author",
    lazy="select" # Valor por defecto
)

# Eager loading: carga posts junto con el autor
posts: Mapped[list["Post"]] = relationship(
    back_populates="author",
    lazy="joined" # Siempre hace JOIN
)
```

7. Ejemplo Completo

```
# models/author.py
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Author(Base):
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)

    # Relación 1:N con Post
    posts: Mapped[list["Post"]] = relationship(
        back_populates="author",
        cascade="all, delete-orphan"
    )
```

```
)
```

```
def __repr__(self) -> str:  
    return f"Author(id={self.id}, name='{self.name}')
```

```
# models/post.py
```

```
from datetime import datetime  
from sqlalchemy import String, Text, ForeignKey  
from sqlalchemy.orm import Mapped, mapped_column, relationship  
from database import Base
```

```
class Post(Base):  
    __tablename__ = "posts"  
  
    id: Mapped[int] = mapped_column(primary_key=True)  
    title: Mapped[str] = mapped_column(String(200))  
    content: Mapped[str] = mapped_column(Text)  
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)  
  
    # Foreign Key  
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))  
  
    # Relación inversa  
    author: Mapped["Author"] = relationship(back_populates="posts")  
  
    def __repr__(self) -> str:  
        return f"Post(id={self.id}, title='{self.title}')
```

✓ Checklist

- ☐ Entiendo qué es una relación 1:N
- ☐ Sé colocar `ForeignKey` en el lado correcto (muchos)
- ☐ Puedo usar `relationship()` con `back_populates`
- ☐ Puedo navegar de parent a children y viceversa
- ☐ Conozco las opciones `cascade` y `lazy`

[Siguiente: Relaciones Muchos a Muchos →](#)